

Name: Swaraj Marathe

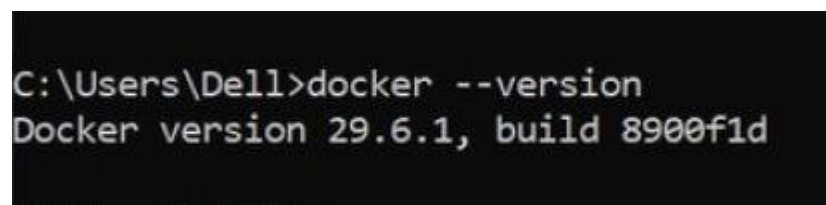
Rollno: MCA2504107

Docker - Container Management

Lab 2 : Docker Container Lifecycle and Management Using Nginx

Step 1: Check Docker Installation

`docker --version`



```
C:\Users\Dell>docker --version
Docker version 29.6.1, build 8900f1d
```

What is happening?

When you type:

`docker --version`

Docker checks whether the **Docker CLI (Command Line Interface)** is installed on your computer.

You

|

| `docker --version`

▼

Docker CLI

|

▼

Displays Version

Example Output

Docker version 28.2.2, build 123456. This command **does not contact Docker Hub or create anything**. It simply checks the Docker program installed on your machine.

Step 2: Search for Nginx

docker search nginx

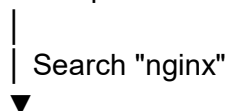
```
C:\Users\Dell>docker search nginx
```

NAME	DESCRIPTION	STARS	OFFICIAL
nginx	Official build of Nginx.	21337	[OK]
nginx/nginx-ingress	NGINX and NGINX Plus Ingress Controllers fo...	121	
nginx/nginx-prometheus-exporter	NGINX Prometheus Exporter for NGINX and NGINL...	52	
nginx/nginx-ingress-operator	NGINX Ingress Operator for NGINX and NGINX P...	4	
nginx/nginxaas-loadbalancer-kubernetes		1	
bitnamicharts/nginx	Bitnami Helm chart for NGINX Open Source	4	
ubuntu/nginx	Nginx, a high-performance reverse proxy & we...	141	
kasmweb/nginx	An Nginx image based off nginx:alpine and in...	9	
rancher/nginx		4	
linuxserver/nginx	An Nginx container, brought to you by LinuxS...	236	
dtagdevsec/nginx	T-Pot Nginx	0	
paketobuildpacks/nginx		0	
vmware/nginx		3	
gluufederation/nginx	A customized NGINX image containing a consu...	1	
cleanstart/nginx	Secure by Design, Built for Speed, Hardened ...	0	
antrea/nginx	Nginx server used for Antrea e2e testing	0	
activestate/nginx	ActiveState's customizable, low-to-no vulner...	0	
intel/nginx		0	
docksal/nginx	Nginx service image for Docksal	1	
geokrety/nginx	Our customized nginx image	0	
circleci/nginx	This image is for internal use	2	
ilios/nginx	Nginx customized to run Ilios along with the...	0	
corpusops/nginx	https://github.com/corpusops/docker-images/	1	
wayofdev/nginx	Nginx Docker image for PHP development. SSL-...	0	
dockette/nginx	Nginx SSL / HSTS / HTTP2	3	

What happens?

Docker connects to **Docker Hub**, which is the official online repository for Docker images.

Your Computer



Docker Hub



Returns Matching Images

Example Output

NAME	DESCRIPTION
nginx	Official build of Nginx
nginxinc/nginx	
bitnami/nginx	

Why?

Docker images are stored online. Before downloading one, you can search for available images.

Think of Docker Hub like the **Google Play Store** or **Apple App Store**, but instead of apps, it stores Docker images.

Step 3: Download Nginx

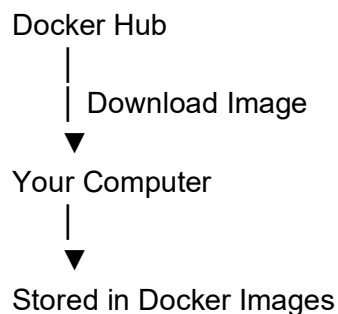
docker pull nginx

```
C:\Users\Dell>docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
45ef33eb91c8: Pull complete
6a24a6ebcf30: Pull complete
0b19be717994: Pull complete
8d2092eabbc5: Pull complete
3b9f3509a826: Pull complete
062e450697fa: Pull complete
39694cde734e: Pull complete
cd64a2895680: Download complete
29456554d205: Download complete
Digest: sha256:c90e4ea5905ee69515d84cd5520e11d204df06cce71a312666cebbbc4b9b68267
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview nginx
```

What happens?

Docker downloads the **Nginx image** from Docker Hub to your computer.



After downloading:

docker images

shows

REPOSITORY	TAG
nginx	latest

Important

At this point:

✗ No container exists.

Only the **image** exists.

Think of it like:

Image = Software Installer

Container = Installed & Running Software

Step 4: View Images

docker images

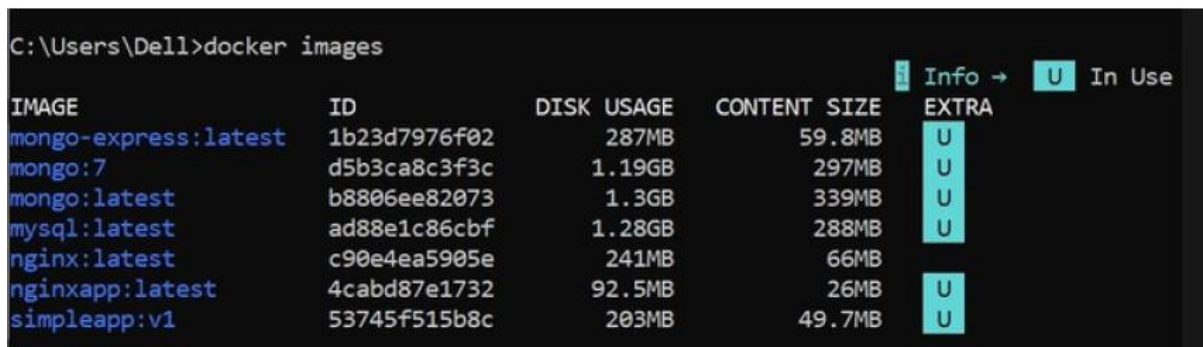


IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
mongo-express:latest	1b23d7976f02	287MB	59.8MB	U
mongo:7	d5b3ca8c3f3c	1.19GB	297MB	U
mongo:latest	b8806ee82073	1.3GB	339MB	U
mysql:latest	ad88e1c86cbf	1.28GB	288MB	U
nginx:latest	c90e4ea5905e	241MB	66MB	U
nginxapp:latest	4cabd87e1732	92.5MB	26MB	U
simpleapp:v1	53745f515b8c	203MB	49.7MB	U

What happens?

Docker shows all downloaded images.

Example

IMAGE ID

nginx

ubuntu

mysql

Nothing is running yet.

You only have installers.

Step 5: Run Nginx

`docker run nginx`

This is one of the most important Docker commands.

```
C:\Users\Dell>docker run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform c
onfiguration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.s
h
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/def
ault.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/d
efault.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/07/14 08:59:22 [notice] 1#1: using the "epoll" event method
2026/07/14 08:59:22 [notice] 1#1: nginx/1.31.2
2026/07/14 08:59:22 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/07/14 08:59:22 [notice] 1#1: OS: Linux 6.6.87.1-microsoft-standard-WSL2
2026/07/14 08:59:22 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/07/14 08:59:22 [notice] 1#1: start worker processes
2026/07/14 08:59:22 [notice] 1#1: start worker process 29
2026/07/14 08:59:22 [notice] 1#1: start worker process 30
2026/07/14 08:59:22 [notice] 1#1: start worker process 31
2026/07/14 08:59:22 [notice] 1#1: start worker process 32
```

What happens internally?

Docker performs several actions:

Step A

Checks whether the image exists.

Is nginx image available?

YES

If not:

Docker automatically performs:

`docker pull nginx`

Step B

Creates a **new container** from the image.

Image



New Container

The image remains unchanged.

Containers are copies of the image.

Step C

Starts the Nginx server.

Container



Nginx Starts

Now Nginx is running inside the container.

Step D

Your terminal becomes attached to the running container.

Terminal



Container Output

Stopping the program using

Ctrl + C

stops the container.

Step 6: Detached Mode

`docker run -d nginx`

What changed?

`-d`

means

Detached Mode

Instead of connecting your terminal to the container:

Terminal

↓

Gets Prompt Back

The container continues running in the background.

Docker returns

`9f3ab2c89c8d...`

This is the **Container ID**.

Step 7: Name the Container

`docker run -d --name mynginx nginx`

Without a name:

Docker generates random names.

Example

`adoring_pike`

`crazy_turing`

`happy_einstein`

Using

```
--name mynginx
```

makes management easier.

Instead of

```
docker stop 98ab123...
```

you simply type

```
docker stop mynginx
```

Step 8: Publish a Port

```
docker run -d --name webserver -p 8080:80 nginx
```

This is the most important networking concept.

Inside the container

Nginx always listens on

Port 80

Container

Nginx

↓

80

But your computer cannot automatically access that internal port.

So Docker creates a mapping.

Computer

8080

↓

Container

80

Meaning

localhost:8080

↓

Container Port 80

Opening

<http://localhost:8080>

actually forwards the request to

Container

↓

Nginx

↓

Port 80

Step 9: List Running Containers

`docker ps`

Shows

Container ID

Image

Status

Ports

Name

Example

mynginx

Up 5 minutes

0.0.0.0:8080->80

Meaning

Host Port

8080

↓

Container Port

80

Step 10: Stop Container

```
docker stop mynginx
```

Docker sends a signal to Nginx.

Please Stop Gracefully

Nginx saves its state and exits cleanly.

Container Status

Running

↓

Exited

Notice:

The **container is not deleted**.

It is only stopped.

Step 11: Start Container Again

```
docker start mynginx
```

Docker does **not create a new container**.

It simply starts the **existing** one.

Stopped Container

↓

Running Container

Step 12: Restart

```
docker restart mynginx
```

Equivalent to

```
docker stop mynginx
```

```
docker start mynginx
```

Step 13: View Logs

```
docker logs mynginx
```

Every application writes messages.

Example

Nginx Started

Request Received

Response Sent

Docker stores these logs.

Useful for debugging.

Step 14: Enter the Container

```
docker exec -it mynginx bash
```

This command opens a shell **inside the running container**.

Your Computer

↓

Docker

↓

Container

↓

Linux Bash

Now commands like

```
pwd
```

```
ls
```

```
cd /usr/share/nginx/html
```

are executed **inside the container**, not on your Windows machine.

This is very useful for troubleshooting or inspecting files.

Step 15: Remove the Container

```
docker rm mynginx
```

Docker deletes the **container only**.

Container

✗ Deleted

The image is still available.

You can create a new container anytime using:

```
docker run nginx
```

Step 16: Remove the Image

```
docker rmi nginx
```

Now Docker removes the **downloaded image** from your local system.

Image

✗ Deleted

If any container still depends on that image, Docker will refuse to remove it until those containers are deleted.